# Parallel Version of w03 Simulator I

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

May 2026.

# What changed?

# From step loop to goroutines

## Basic Version

- single-threaded simulator
- controlled by a `Step()` loop
- one queued message processed per step
- deterministic for a fixed seed
- trace is a sequence of discrete scheduler steps

## Concurrent Version

- goroutine-based runtime
- processes run independently
- communication uses channels
- controlled by scheduling, context cancellation, and timeouts
- trace shows real goroutine interleaving

## Key Takeaway

The protocol examples stay the same, but the **execution model** changes completely.

# Same algorithms, but different execution model

- The basic version was designed to introduce and isolate the **core basic abstractions**.
  - one step at a time
  - one message at a time
  - easier to reason about correctness
- The concurrent version is a **goroutine-based implementation**.
  - multiple processes active at once
  - messages flow asynchronously
  - closer to how real concurrent systems feel
- Protocol logic is still same.
- What changes is **how the protocol is executed**.

# New Process Model

# Why do we need to change the Process Model?

- The basic simulator modeled a process as a small deterministic state machine.
- That model was ideal for:
  - getting to know abstractions
  - reasoning about traces
  - correctness proofs
- But real concurrent systems do not execute as isolated atomic steps.
- Real processes:
  - run continuously
  - wait for messages asynchronously
  - communicate concurrently

**Goal**

Move from a passive process model to an active goroutine-based actor model.

# Basic Process

## Basic process interface

```
type Process interface {
    ID() ProcessID
    Handle(msg Message) []Message
}
```

- `Handle(...)` processes exactly one message
- execution is atomic from the simulator perspective
- returned messages are collected by the runtime
- the runtime controls all scheduling
- process behaves like a deterministic state machine

# Concurrent Process

## Concurrent process interface

```
type Process interface {
    ID() ProcessID
    Run(
        ctx context.Context, // controls shutdown
        inbox <-chan Message, // receives messagesreceives messages
        send func(Message) // sends messages out
    )
}
```

- a process is now a long-running goroutine
- messages arrive asynchronously through `inbox`
- outgoing messages are emitted using `send(...)`
- `ctx` controls cancellation and shutdown
- process is no longer a single atomic step, it is now an active concurrent actor

# Differences

## Basic Version

- passive process
- runtime drives execution
- one message processed at a time
- deterministic scheduling
- easier correctness reasoning

## Concurrent Version

- active goroutine
- process drives its own execution
- asynchronous message handling
- interleaving by Go scheduler
- more realistic concurrency

# Compatibility Layer: `WrapHandler()`

## Compatibility adapter

```
type Handler interface {
    ID() ProcessID
    Handle(msg Message) []Message
}
func WrapHandler(h Handler) Process
```

- many examples were originally written using `Handle(...)`
- rewriting every example would be unnecessary
- `WrapHandler()` bridges old handlers into the new runtime

## Key Idea

The adapter preserves compatibility between the old synchronous model and the new concurrent execution environment.

# How `WrapHandler()` Works?

1. wait for a message on the inbox channel
2. call:

   `Handle(msg)`
3. collect returned replies
4. emit each reply through:

   `send(reply)`
5. continue until:
   - context is cancelled
   - inbox is closed

**Transformation**

`one message in - replies out, becomes run forever - receive asynchronously - send asynchronously`

# Example: Counter Adaptation

- `CounterHandler` and `WorkerHandler` still use:

  `Handle(msg)`

- They are reused through:

  `process.WrapHandler(...)`

- The protocol logic remains almost unchanged, but there is:

## New concurrency concern

Shared state must now be concurrency-safe:

- `atomic.AddInt64(...)`
- `atomic.LoadInt64(...)`

## Key Idea

The algorithm stayed the same, but concurrency introduced synchronization concerns.

# New Runtime Architecture

# Concurrent Runtime Architecture

## Core runtime components

```
type Runtime struct {
    processes map[ProcessID]*procEntry // processes with their inbox channels
    network chan Message // shared transport for outgoing messages
    link link.Link
    failures failures.FailureInjector
    trace *Trace
    checkers []Checker
    interceptor func(Message) Message
    lastDelivery atomic.Value // last successful delivery for idle detection
    maxDuration time.Duration
    idleTimeout time.Duration // stops if no deliveries happen for this duration
    retransmitInterval time.Duration}
```

- no more one central queue, now runtime coordinates active goroutines using channels, timeouts, and cancellation.

# Concurrent Runtime Components

## Process entries

- each process is stored together with its own inbox channel
- `procEntry = process + inbox`

## Network channel

- `network chan Message` simulates the shared transport layer
- all outgoing messages enter the network before delivery

## Runtime goroutines

- process goroutines execute `Run(ctx, inbox, send)`
- router goroutine moves messages from network to inboxes
- retransmitter goroutine periodically injects retry traffic
- idle monitor cancels execution when the system becomes inactive

# Message Path in the Concurrent Runtime

1. process calls `send(msg)`
2. runtime logs `SEND`
3. failure model may alter outgoing message
4. link `Send(...)` decides what survives
5. surviving messages are placed on `network`
6. router reads from `network`
7. interceptor, failures, and link `Receive(...)` are applied
8. message is delivered to recipient's inbox
9. recipient goroutine receives it from `inbox`

## Key Difference

In the basic runtime, the runtime directly calls `Handle(msg)`. In the concurrent runtime, delivery means sending the message into a process inbox.

# Concurrent Runtime Lifecycle

1. create context with `maxDuration`
2. start one goroutine per process
3. start router goroutine
4. start retransmitter goroutine
5. start idle monitor goroutine
6. wait for goroutines using `sync.WaitGroup`
7. run liveness checkers at the end
8. print trace summary

## Termination

The simulation ends when:

- `maxDuration` expires, or
- no deliveries happen for `idleTimeout`

# Queue vs Network Channel

## Basic Version

- queue implemented as:

```
[]Message
```

- runtime manually selects messages
- communication is simulated

## Concurrent Version

- shared transport implemented as:

```
chan Message
```

- goroutines communicate asynchronously
- communication is done with real Go concurrency

## Key Shift

A passive queue becomes an active concurrent communication mechanism.

# Router and Retransmitter Goroutines

## Router Goroutine

- listens on the shared network channel
- routes messages to the correct process inbox
- replaces centralized queue dequeue logic

## Retransmitter Goroutine

- periodically asks links for retransmissions
- injects retry traffic back into the network
- enables stubborn-link behavior concurrently

## Key Idea

Responsibilities previously handled inside `Step()` become independent concurrent workers.

# Termination Model Change

## Basic Runtime

- execution bounded by:

`maxSteps`

- simulator stops after enough scheduler turns
- deterministic stopping point

## Concurrent Runtime

- execution bounded by:

`maxDuration`

- idle monitor detects inactivity
- context cancellation shuts down goroutines

## Key Shift

Termination changes from step-count based execution to time and activity based coordination.

# Runtime Philosophy

## Basic Runtime

```
runtime = scheduler + execution engine
```

## Concurrent Runtime

```
runtime = coordinator of concurrent components
```

- same distributed abstractions
- same protocol ideas
- different execution semantics

## Big Picture

The runtime is where the shift from abstract simulation to realistic concurrent execution becomes visible.